UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

Semester 01 2025/2026

Cinema Booking System

Subject : Database Programming (SECP3623)

Section : Section 1

Task : Project II

Due : 19th January 2026 (10%)

Name	Matric No.
Choh Jing Yi	A23CS0296
Chew Chiu Xian	A23CS0061
Neo Li Xin	A23CS0253

Video presentation link:

https://youtu.be/iifowYX_2yA

Index

1.0 Introduction **3**
1.1 Description of Domain **3**
1.2 Project Objectives **3**
1.3 Team Members and Roles **3**
2.0 NoSQL Data Model **4**
2.1 List and Explanation of Collections **4**
2.2 Example JSON Documents **4**
2.3 Explanation of Embedding vs Referencing **6**
2.4 Data Types Used **7**
3.0 CRUD Implementation Summary **8**
3.1 Explanation of Insert, Query, Update, Delete Operations **8**
3.2 Screenshots of MongoDB Compass or Shell Execution **9**
3.3 Projections and Query Tests **15**
4.0 Advanced Operations **17**
4.1 Indexing Creation Example **17**
4.2 Aggregation Pipeline Examples **17**
4.3 Sorting Demonstrations **19**
4.4 Index Query Performance Improvements **21**
4.5 Challenges **22**
5.0 Security and Limitations **22**
5.1 Basic Access Controls **22**
5.2 Injection Risk **22**
5.3 Database Constraints and Limitations **23**
6.0 Teamwork **23**
7.0 Conclusion **25**

1.0 Introduction

1.1 Description of Domain

This project involves designing a backend for a Cinema Booking System using MongoDB to manage movie schedules, seat reservations, and customer data efficiently. In the real world cinema industry, customers require real-time access to movie schedules, seat availability, and ticketing services. The system uses four main collections: movies, screenings, users and bookings to perform essential operations such as managing bookings, updating seat inventory and processing transactions.

1.2 Project Objectives

The primary objectives of this database system are:

- Design a NoSQL backend system that handles cinema data to efficiently manage bookings and user profiles.
- Implement CRUD operations (insert, find, update, delete), indexing, and aggregation workflows
- Perform data analytics allow the system to generate business insights such as calculating total revenue per movie and identifying the most popular film genres

1.3 Team Members and Roles

Member	**Role**
Choh Jing Yi	**Group Leader & Database Architect:** Responsible for the overall project coordination and the structural design of the database. He was also involved in defining the collection schemas and managed the compilation of the final report.
Chew Chiu Xian	**Backend Developer (CRUD Operations):** Focused on the functional implementation of the system. His primary role involved writing the scripts for data insertion and ensuring that

	all Create, Read, Update, and Delete (CRUD) operations functioned correctly.
Neo Li Xin	**Analytics Engineer (Advanced Operations):** In charge of system optimization and business intelligence. He also developed the aggregation pipelines for data analysis, implemented indexing strategies for performance, and assessed system security.

2.0 NoSQL Data Model

2.1 List and Explanation of Collections

1) **movies**: This collection serves as the central catalog for all films shown in the cinema storing some details such as title, language and base price. It utilizes an Array to store multiple genres per film and Data Types like Date for release date of movies.
2) **bookings**: This collection records the transactional history of ticket purchases, the booking status and payment details of every reservation. It demonstrates references to link users and screenings, an array for selected seat numbers, and Date data types for the booking timestamp.
3) **screenings**: This collection manages specific showtimes and seat inventory for each hall. It references the parent movie.There is an array of to represent the status of individual seats in the seat map and Date data types for scheduling.
4) **users**: This collection stores customer profiles and membership information to support loyalty programs and user authentication.

2.2 Example JSON Documents

1. Movies Collection

```
{
 "_id": "MOV001",
 "title": "Zombie Apocalypse: The Beginning",
 "genres": [
  "Horror",
  "Thriller",
  "Action"
 ],
```

```
 "duration": 120,
 "language": "English",
 "releaseDate": "2025-12-15T00:00:00.000Z",
 "basePrice": 20
}
```


2. Screenings Collection


```
{
 "_id": "SCR101",
 "movieId": "MOV001",
 "hall": "Hall A",
 "startTime": {
  "$date": "2026-01-20T10:00:00.000Z"
 },
 "price": 22,
 "seatMap": [
  {
   "seatNo": "A1",
   "status": "booked"
  },
  {
   "seatNo": "A2",
   "status": "booked"
  },
  {
   "seatNo": "A3",
   "status": "available"
  },
  {
   "seatNo": "A4",
   "status": "available"
  },
  {
   "seatNo": "A5",
   "status": "available"
  }
 ]
}
```


3. Users Collection

```
{
 "_id": "USER_001",
 "name": "Neo Li Xin",
 "email": "neo@gmail.com",
 "membershipLevel": "Gold",
 "loyaltyPoints": 150
}
```


4. Bookings Collection


```
{
 "_id": "BKG501",
 "userId": "USER_001",
 "screeningId": "SCR101",
 "seats": [
  "A1",
  "A2"
 ],
 "totalPrice": 44,
 "paymentMethod": "Credit Card",
 "bookingDate": "2026-01-18T02:30:00.000Z",
 "status": "Confirmed"
}
```


2.3 Explanation of Embedding vs Referencing

Method	Implementation in Project	Justification & Benefits
Embedding	seatMap inside screening	1. **Increase read performance.** The most common query is showtime. Embedding enables the app to access the schedule and the availability of the seat in a single database. 2. **Atomic Update:** We can atomically update the status

		of a particular seat by asking it to be locked so that it can be sold within the screening document, so that data is consistent when there is heavy traffic.
Referencing	userId and screeningId inside bookings	1. **Scalability**: Embedding the bookings in a User document will eventually reach the 16MB document limit of MongoDB when the user views a large number of movies. Referencing avoids this. 2. **Data Consistency:** Screening is used by a large number of users. With the help of the screeningId, any change in the showtime is updated in real-time to all the booked users without the need to update thousands of booking records.

2.4 Data Types Used

Data Type	Field Examples	Usage
String	title, hall, email, _id	Stores text-based data
Integer	duration, loyaltyPoints	Stores whole numbers
Double	basePrice, price, totalPrice	Stores floating point numbers for values requiring decimal precision
Date	releaseDate, startTime, bookingDate	Stores dates and times

Array	genres, seats	This allows a movie to have multiple categories without creating a separate lookup table
Object (Embedded Document)	seatMap items	We used Embedded Documents for the seatMap. We store each seat number and status inside a parent document,seatMap

3.0 CRUD Implementation Summary

3.1 Explanation of Insert, Query, Update, Delete Operations

a. Insert operations

We use insert operations to add the initial data records.

- **insertOne():** We used this to add a single new Users profile or a new Movies release.
- **insertMany():** We used this to import the initial data record of Movies and Screenings which is more efficient than inserting documents one by one.

b. Query operations

We use query operations to retrieve specific data based on user search criteria.

- Filter using **$regex**

We implemented **$regex** to perform flexible text searches. This allows the system to find movies even if the user only remembers part of the title. For example,customer search for "Apocalypse" finds "Zombie Apocalypse: The Beginning").

- Filter using **$nor**

We used the **$nor** operator to exclude specific categories. This query finds movies that are neither "Action" nor "Thriller" effectively filtering out uninterested movies for audiences.

- Filter using **$in**

We used the **$in** operator to find movies that belong to a specific set of genres. This allows users to search for multiple categories like "Horror" or "Comedy" in a single search.

- Range Filter using **$and, $gte, $lte**

We combined **$and** with **$gte** and **$lte** to find screenings within a specific price budget. This helps customers find tickets with their budget.

c. Update Operations

We use update operations for modifying existing records such as processing bookings or managing inventory.

- **updateOne()** and **$set**: We used updateOne() with the $set operator to change the status of a booking from "Pending" to "Confirmed" after a successful payment.
- **$push**: We used $push to add new genres to a movie.

d. Delete Operations

We use delete operations to remove unwanted data records. This ensures the database remains clean and stores only valid data.

- **deleteOne() :** We implemented **deleteOne()** to remove one document that matches a specific condition in a single operation. For this project, we implemented **deleteOne()** to remove booking records that were "Cancelled" by the user. This ensures the database remains clean and stores only valid active transactions.

- **deleteMany() :** We implemented **deleteMany()** to efficiently remove multiple documents that match a specific condition in a single operation. For this project, we implement removing all screenings associated with a specific movie if that film is suddenly pulled from the schedule.

3.2 Screenshots of MongoDB Compass or Shell Execution

a. Insert operations

- **insertOne():**

```
> db.movies.insertOne({
    "_id": "MOV006",
    "title": "Future World",
    "genres": ["Sci-Fi", "Drama"],
    "duration": 130,
    "language": "English",
    "releaseDate": new Date("2026-02-01"),
    "basePrice": 22.0
    })
< {
    acknowledged: true,
    insertedId: 'MOV006'
  }
```

- **insertMany():**

```
cinema_booking_db> db.movies.insertMany([
                  {
                    "_id": "MOV001",
                    "title": "Zombie Apocalypse: The Beginning",
                    "genres": ["Horror", "Thriller", "Action"],
                    "duration": 120,
                    "language": "English",
                    "releaseDate": ISODate("2025-12-15T00:00:00.000Z"),
                    "basePrice": 20.0
                  },
                  {
                    "_id": "MOV002",
                    "title": "Tech Titans: Silicon Valley",
                    "genres": ["Documentary", "Drama"],
                    "duration": 95,
                    "language": "English",
                    "releaseDate": ISODate("2026-01-01T00:00:00.000Z"),
                    "basePrice": 15.0
                  },
                  {
                    "_id": "MOV003",
                    "title": "Galactic Wars",
                    "genres": ["Sci-Fi", "Adventure"],
                    "duration": 145,
                    "language": "English",
                    "releaseDate": ISODate("2025-11-20T00:00:00.000Z"),
                    "basePrice": 25.0
                  },
```

```
  {
    "_id": "MOV004",
    "title": "Family Fun Day",
    "genres": ["Animation", "Comedy", "Family"],
    "duration": 90,
    "language": "Malay",
    "releaseDate": ISODate("2025-12-25T00:00:00.000Z"),
    "basePrice": 12.0
  },
  {
    "_id": "MOV005",
    "title": "Shadows of the Past",
    "genres": ["Mystery", "Thriller"],
    "duration": 110,
    "language": "Korean",
    "releaseDate": ISODate("2026-01-05T00:00:00.000Z"),
    "basePrice": 18.0
  }
]);
```

b. Query operations

- Filter using **$regex**

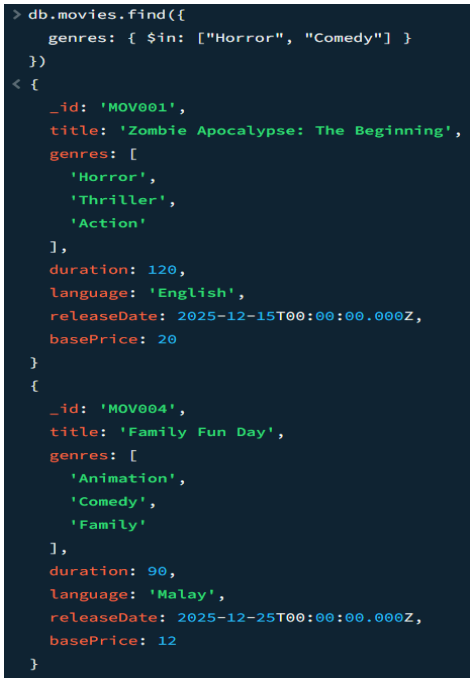
```
> db.movies.find({
    title: { $regex: "Apocalypse", $options: "i" }
  });
< {
    _id: 'MOV001',
    title: 'Zombie Apocalypse: The Beginning',
    genres: [
      'Horror',
      'Thriller',
      'Action'
    ],
    duration: 120,
    language: 'English',
    releaseDate: 2025-12-15T00:00:00.000Z,
    basePrice: 20
  }
```

- Filter using **$nor**

```
> db.movies.find({
    $nor: [
      { genres: "Action" },
      { genres: "Thriller" }
    ]
  })
< {
    _id: 'MOV006',
    title: 'Future World',
    genres: [
      'Sci-Fi',
      'Drama'
    ],
    duration: 130,
    language: 'English',
    releaseDate: 2026-02-01T00:00:00.000Z,
    basePrice: 22
  }
  {
    _id: 'MOV002',
    title: 'Tech Titans: Silicon Valley',
    genres: [
      'Documentary',
      'Drama'
    ],
    duration: 95,
    language: 'English',
    releaseDate: 2026-01-01T00:00:00.000Z,
    basePrice: 15
  }
```

```
{
  _id: 'MOV003',
  title: 'Galactic Wars',
  genres: [
    'Sci-Fi',
    'Adventure'
  ],
  duration: 145,
  language: 'English',
  releaseDate: 2025-11-20T00:00:00.000Z,
  basePrice: 25
}
{
  _id: 'MOV004',
  title: 'Family Fun Day',
  genres: [
    'Animation',
    'Comedy',
    'Family'
  ],
  duration: 90,
  language: 'Malay',
  releaseDate: 2025-12-25T00:00:00.000Z,
  basePrice: 12
}
```

- Filter using **$in**



```
> db.movies.find({
    genres: { $in: ["Horror", "Comedy"] }
  })
< {
    _id: 'MOV001',
    title: 'Zombie Apocalypse: The Beginning',
    genres: [
      'Horror',
      'Thriller',
      'Action'
    ],
    duration: 120,
    language: 'English',
    releaseDate: 2025-12-15T00:00:00.000Z,
    basePrice: 20
  }
  {
    _id: 'MOV004',
    title: 'Family Fun Day',
    genres: [
      'Animation',
      'Comedy',
      'Family'
    ],
    duration: 90,
    language: 'Malay',
    releaseDate: 2025-12-25T00:00:00.000Z,
    basePrice: 12
  }
```

- Filter using **$and, $gte, $lte**

```
> db.screenings.find({
    $and: [
      { price: { $gte: 25 } },
      { price: { $lte: 30 } }
    ]
  })
< {
    _id: 'SCR102',
    movieId: 'MOV001',
    hall: 'Hall B',
    startTime: 2026-01-20T13:00:00.000Z,
    price: 25,
    seatMap: [
      {
        seatNo: 'A1',
        status: 'booked'
      },
      {
        seatNo: 'A2',
        status: 'available'
      },
      {
        seatNo: 'A3',
        status: 'maintenance'
      },
      {
        seatNo: 'A4',
        status: 'available'
      },
      {
        seatNo: 'A5',
        status: 'available'
```

c. Update Operations

- **updateOne()** and **$set**

```
> db.bookings.updateOne(
    { "_id": "BKG504" },
    { $set: { "status": "Confirmed" } }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

```
> db.bookings.find({_id:"BKG504"})
< {
    _id: 'BKG504',
    userId: 'USER_001',
    screeningId: 'SCR104',
    seats: [
      'C5'
    ],
    totalPrice: 40,
    paymentMethod: 'Credit Card',
    bookingDate: 2026-01-21T01:00:00.000Z,
    status: 'Confirmed'
  }
```

- **$push**

```
> db.movies.updateOne(
    { "_id": "MOV004" },
    { $push: { genres: "Adventure" } }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

```
> db.movies.find({_id:"MOV004"})
< {
    _id: 'MOV004',
    title: 'Family Fun Day',
    genres: [
      'Animation',
      'Comedy',
      'Family',
      'Adventure'
    ],
    duration: 90,
    language: 'Malay',
    releaseDate: 2025-12-25T00:00:00.000Z,
    basePrice: 12
  }
```

d. Delete Operations

- **deleteOne()**

```
> db.bookings.deleteOne({
    "status": "Cancelled"
  })
< {
    acknowledged: true,
    deletedCount: 1
  }
```

- **deleteMany()**

```
> db.screenings.deleteMany({
    movieId: "MOV005"
  })
< {
    acknowledged: true,
    deletedCount: 1
  }
```

3.3 Projections and Query Tests

Test 1: Display Movie Catalog

Scenario: The user is browsing the "Now Showing" list. They only want to see the Movie Title and Genres. We exclude the _id and other details to keep the response lightweight.

```
> db.movies.find(
    {},
    {
      title: 1,
      genres: 1,
      _id: 0
    }
  )
< {
    title: 'Future World',
    genres: [
      'Sci-Fi',
      'Drama'
    ]
  }
  {
    title: 'Zombie Apocalypse: The Beginning',
    genres: [
      'Horror',
      'Thriller',
      'Action'
    ]
  }
  {
    title: 'Tech Titans: Silicon Valley',
    genres: [
      'Documentary',
      'Drama'
    ]
  }
```

```
{
  title: 'Galactic Wars',
  genres: [
    'Sci-Fi',
    'Adventure'
  ]
}
{
  title: 'Family Fun Day',
  genres: [
    'Animation',
    'Comedy',
    'Family',
    'Adventure'
  ]
}
{
  title: 'Shadows of the Past',
  genres: [
    'Mystery',
    'Thriller'
  ]
}
```

Test 2: Showtimes & Pricing (Exclude Seat Map)

Scenario: A customer wants to check the price and time for "Galactic Wars". We return the hall, start time, and price but exclude the large seatMap array to reduce the result size significantly.

```
> db.screenings.find(
    { movieId: "MOV003" },
    {
      hall: 1,
      startTime: 1,
      price: 1,
      _id: 0
    }
  )
< {
    hall: 'IMAX 1',
    startTime: 2026-01-21T06:00:00.000Z,
    price: 35
  }
```

4.0 Advanced Operations

4.1 Indexing Creation Example

i) Single-field Index

```
> db.movies.createIndex({ title: 1 });
< title_1
```

This index speeds up search queries where the application looks up a movie by its name (e.g., db.movies.find({ title: "Galactic Wars" })), preventing the database from scanning every single document.

ii) Compound Index

```
> db.screenings.createIndex({ movieId: 1, startTime: 1 });
< movieId_1_startTime_1
```

This optimizes queries that filter by a specific movie and then sort the results by time (e.g., "Show me all showtimes for *Zombie Apocalypse* today").

4.2 Aggregation Pipeline Examples

i) Total Revenue by Payment Method

Calculate how much money was made from each payment type (Credit Card, E-Wallet), but only for "Confirmed" bookings.

```
> db.bookings.aggregate([
    {
      $match: { status: "Confirmed" }
    },
    {
      $group: {
        _id: "$paymentMethod",
        totalRevenue: { $sum: "$totalPrice" },
        count: { $sum: 1 }
      }
    },
    {
      $sort: { totalRevenue: -1 }
    }
  ]);
< {
    _id: 'E-Wallet',
    totalRevenue: 60,
    count: 2
  }
  {
    _id: 'Credit Card',
    totalRevenue: 44,
    count: 1
```

This returns a summary that shows which payment method generates the most revenue.

ii) Top 3 Most Popular Genres

Determine which genres have the most movies in the database.

```
> db.movies.aggregate([
    {
      $unwind: "$genres"
    },
    {
      $group: {
        _id: "$genres",
        movieCount: { $sum: 1 }
      }
    },
    {
      $sort: { movieCount: -1 }
    },
    {
      $limit: 3
    }
  ]);
< {
    _id: 'Thriller',
    movieCount: 2
  }
  {
    _id: 'Animation',
    movieCount: 1
  }
  {
    _id: 'Drama',
    movieCount: 1
  }
```

This helps cinema managers understand the distribution of movie types in their catalog.

4.3 Sorting Demonstrations

i) Ascending

List movies from shortest duration to longest.

```
> db.movies.find({}, { title: 1, duration: 1 }).sort({ duration: 1 });
< {
    _id: 'MOV004',
    title: 'Family Fun Day',
    duration: 90
  }
  {
    _id: 'MOV002',
    title: 'Tech Titans: Silicon Valley',
    duration: 95
  }
  {
    _id: 'MOV005',
    title: 'Shadows of the Past',
    duration: 110
  }
  {
    _id: 'MOV001',
    title: 'Zombie Apocalypse: The Beginning',
    duration: 120
  }
  {
    _id: 'MOV003',
    title: 'Galactic Wars',
    duration: 145
  }
```

ii) Descending

Find the most loyal users (highest loyalty points).

```
> db.users.find({}, { name: 1, loyaltyPoints: 1 }).sort({ loyaltyPoints: -1 });
< {
    _id: 'USER_001',
    name: 'Neo Li Xin',
    loyaltyPoints: 150
  }
  {
    _id: 'USER_004',
    name: 'John Doe',
    loyaltyPoints: 65
  }
  {
    _id: 'USER_002',
    name: 'Choh Jing Yi',
    loyaltyPoints: 50
  }
  {
    _id: 'USER_003',
    name: 'Chew Chiu Xian',
    loyaltyPoints: 10
  }
```

4.4 Index Query Performance Improvements

MongoDB indexes can be compared to the index of a book, and it enables the system to find data without having to read through all pages. In case of lack of an index, the database will conduct a Collection Scan (COLLSCAN) i.e. it has to scan all the documents under the collection to get a match and this is very inefficient with large datasets.

1. Without Indexing:

Incidentally, when we execute a query such as db.movies.find({ title: "Galactic Wars" }) without an index, MongoDB will have to search all documents in the movies collection to determine whether the title matches. The increase in data will make this process longer as it uses a lot of processing power (CPU) and disk I/O.

2. With Indexing:

Once we index the title field (db.movies.createIndex({ title: 1 })), the MongoDB will generate a different and sorted object that will only store the values of the title field and

an address to the location of the real document in the disk. This also greatly enhances the speed at which data can be accessed since the database is able to find a particular document instantly without having to search through all the records in the collection.

4.5 Challenges

One of the major problems that were faced in the process of implementing the advanced operations, as the atomic updates were to be managed with the deeply nesting document structures, particularly, with the seatMap array in the Screenings collection. As opposed to relational databases where the availability of a particular seat would generally be represented within a separate row, our denormalized NoSQL design would necessitate a modification of the status of a given seat object within an array without replacing an entire document. This required the usage of the positional operator of MongoDB to locate and update the specific array element defined by the seat number, which required the accurate usage of the positional operator called $. Also, the change of the traditional relational design to the document-oriented design was a design problem since it had to be planned well to balance between data redundancy and query throughput; i.e., deciding when to refer to data to provide consistency in the booking and user history records.

5.0 Security and Limitations

5.1 Basic Access Controls

To make sure that the cinema ticketing system is not compromised, basic access control needs to be implemented based on the responsibilities of the users. In a production environment, it is recommended that the use of the database root user be avoided and that Role-Based Access Control (RBAC) be adopted. For example, a "CinemaManager" role can be allowed read-write privileges for the movies and screen times arrays, while a "TicketAgent" role may have read-only privileges for ticket validation. Besides that, network security practices such as TLS/SSL encryption must be enforced for the protection of confidential customer information such as names and payment identifiers when transmitted between the application and MongoDB.

5.2 Injection Risk

The application is vulnerable to NoSQL injection vulnerabilities, which are different from traditional SQL injection vulnerabilities. However, they still pose a significant threat. The

vulnerability exists because the application accepts user input without validating the user input properly. An example would be when an attacker enters a malicious BSON query operator into a login field or a movie search bar. The attacker may enter an operator such as { "$ne": null } in place of a valid string. If the backend were to directly process the input without performing validation, this could allow the attacker to bypass authentication or to have access to the entire user database. Therefore, it is essential that the application layer validates all input types to ensure that sensitive data such as passwords (or movie titles) is treated as strictly a string and is never treated as an executable JSON object.

5.3 Database Constraints and Limitations

While MongoDB allows for the flexibility of schemas, it also has some limits compared to relational database systems. One of the main constraints is that MongoDB does not enforce foreign keys strictly, meaning that if a document containing information about a movie is deleted from the movies collection, any associated screening documents remain, but the application must manage orphaned documents manually. Also, MongoDB has a maximum document size of 16MB. The benefits of embedding the seat map within the screening document create excellent performance, but we must ensure that our data structure remains as slim as possible so that we do not hit the limit with either a large cinema or a lot of metadata about a movie. Furthermore, while MongoDB does support transactions, it is more difficult and costly to achieve ACID compliance when multiple documents are used, for example when updating a seat's status and creating a booking record at the same time. Thus, all implementations must be thought out well to avoid data inconsistencies in times of high transaction attempts.

6.0 Teamwork

Each group member completed their assigned tasks through effective collaboration and communication. Every member contributed to both the implementation and the documentation of the project. The roles and learning outcomes are as follows:

CHOH JING YI - GROUP LEADER (Database Architect)

As the group leader, I held regular team discussions, organized task distribution, and ensured all project requirements were met before submission. I was responsible for the **NoSQL Data Modelling** and the **Introduction**, acting as the bridge between the conceptual design and the technical implementation. My tasks included defining the domain scope, designing the JSON

schema for the 4 collections, and making critical architectural decisions regarding **Embedding vs. Referencing** to suit our specific use case.

I learned how to design a flexible NoSQL structure that accommodates hierarchical data efficiently and how to effectively lead a team to merge different technical components into a unified system.

CHEW CHIU XIAN - BACKEND DEVELOPER (CRUD Operations)

My primary role is focused on the **CRUD Implementation**, ensuring the database could handle core transactional operations and maintain data integrity. I was responsible for populating the database with realistic sample data using insertMany and verifying that the data types matched our schema definitions. I implemented the important data manipulation logic, including querying with filters ($in, $gte), performing atomic updates on embedded arrays, and deleting records. Furthermore, I documented the standard operating procedures for data entry to ensure consistency.

I gained a strong understanding of MongoDB's query language, specifically how to manipulate nested documents and arrays without compromising data integrity during update operations.

NEO LI XIN - ANALYTICS ENGINEER (Advanced Operations)

I developed the **Advanced Operations** and **Security** sections of the system, focusing on extracting value from our stored data. I designed complex **Aggregation Pipelines** to generate business insights, such as calculating total revenue per movie and identifying high-value customers using stages like $group. Besides that, I implemented **Indexing strategies** to optimize query performance and analyzed the system's security risks regarding NoSQL injection.

I learned how to utilize the Aggregation Framework to perform complex data transformations server-side and understood the critical role of indexing and security protocols in building a robust database application.

7.0 Conclusion

Overall, this project was able to show how to design and implement a scalable NoSQL backend of a Cinema Booking System with use of MongoDB. The development of a document-based data model has allowed the team to maintain the complicated, hierarchical data types, like embedded

seat maps and referenced booking history, which was more flexible than the previous relational databases. We learned the practical knowledge of server-side data analytics and query optimization through the implementation of complete CRUD operations and the creation of more complex data aggregation pipelines. The introduction of single-field and compound indexes only underlined the importance of the performance tuning in the context of the booking query and revenue computation that must be efficient when under load.

Nevertheless, certain drawbacks of the process of its development became especially evident when it came to data integrity, and atomic updates of deeply nested arrays, i.e., changing the status of certain seats in a screening document. Switching the relational mental state to denormalized NoSQL structure also demanded advanced planning to find the balance between the data redundancy and speed of the query. In the future, the system can be made much better through the creation of a reacting frontend system to display the booking process and introduction of more stringent security measures, including Role-Based Access Control (RBAC), as a way to prevent NoSQL injection attacks. Also, next versions must consider the sharding techniques to address the horizontal scaling so that the system can be performant when the amount of the booking data increases.